6 CPU 特性用于性能分析

性能分析的最终目标是识别性能瓶颈并定位与之相关的代码部分。不幸的是，没有预先确定的步骤可以遵循，因此可以采取许多不同的方法。

通常，对应用程序进行分析可以快速洞察应用程序的热点。有时这是开发人员需要做的所有事情，以修复性能效率问题。特别是高层次的性能问题通常可以通过分析来揭示。例如，考虑一种情况，当你刚刚对应用程序中的函数`foo`进行了更改，突然看到性能显著下降。因此，你决定对应用程序进行分析。根据你对应用程序的心理模型，你预计`foo`是一个冷函数，它不会出现在热函数的前 10 名列表中。但是当你打开分析结果时，你看到它比以前消耗了更多的时间。你很快意识到你在代码中犯了一个错误，并修复了它。如果所有性能工程问题都那么容易解决，这本书就不会存在了。

当你开始努力从应用程序中挤出最后一点性能时，最基本的热点列表是不够的。除非你有一个水晶球或者对整个 CPU 有一个准确的模型，否则你需要额外的支持来理解性能瓶颈是什么。然而，在本章中使用所呈现的信息之前，请确保你正在尝试优化的应用程序没有重大的性能缺陷。因为如果它有，使用 CPU 性能监控特性进行低级调整是没有意义的。这可能会误导你的方向，而不是修复真正的高层次性能问题，你将只是在调整糟糕的代码，这纯粹是浪费时间。

一些开发人员依赖于他们的直觉，并进行随机实验，尝试强制各种编译器优化，比如循环展开、向量化、内联等等。确实，有时你可以很幸运，并从你的同事那里得到一些赞美，甚至可能在你的团队中声称一个非官方的性能大师的头衔。但通常，你需要有很好的直觉和运气。在这本书中，我们不教你如何变得幸运。相反，我们展示了在实践中被证明有效的方法。

现代 CPU 不断获得新的特性，以不同的方式增强性能分析。使用这些特性可以大大简化发现低级问题，如缓存未命中、分支预测错误等。在本章中，我们将看看现代 CPU 上可用的一些硬件性能监控功能。不同厂商的处理器不一定具有相同的功能集。在本章中，我们将重点关注 Intel、AMD 和 ARM 处理器中可用的性能监控功能。RISC-V 生态系统还没有成熟的性能监控基础设施，所以我们在这里不会涵盖它。

- **自顶向下微架构分析** (TMA) 方法论，讨论在 Section 6.1 中。这是一种强大的技术，用于识别程序对 CPU 微架构的无效使用。它描述了工作负载的瓶颈，并允许定位源代码中发生瓶颈的确切位置。它抽象了 CPU 微架构的复杂性，即使对于没有经验的开发者来说也相对容易使用。
- **最后分支记录** (LBR)，讨论在 Section 6.2 中。这是一种机制，它在执行程序的同时连续记录最近的分支结果。它用于收集调用栈，识别热分支，计算单个分支的误预测率等。
- **基于处理器事件的采样** (PEBS)，讨论在 Section 6.5 中。这是一个增强采样的特性。其主要好处包括：降低采样的开销；并提供“精确事件”功能，这使得能够精确定位导致特定性能事件的确切指令。
- **Intel 处理器跟踪** (PT)，在附录 D 中讨论。它是一个记录和重构程序执行的功能，每个指令都有一个时间戳。它的主要用途是事后分析和根除性能故障。

Intel PT 特性在附录 D 中介绍。Intel PT 本应成为性能分析的“终极解决方案”。由于其低运行时开销，它是一个非常强大的分析特性。但事实证明，它在性能工程师中并不受欢迎。部分原因是工具支持不成熟，部分原因是在许多情况下它过于复杂，而且使用采样分析器更容易。此外，它产生大量数据，对于长时间运行的工作负载来说并不实用。

上述特性提供了从 CPU 角度对程序效率的洞察。在下一章中，我们将讨论分析工具如何利用它们提供多种类型的性能分析。

6.1 自顶向下微架构分析

自顶向下微架构分析（TMA）方法论是一种非常强大的技术，用于识别程序中的 CPU 瓶颈。这是一种健壮且正式的方法论，即使对于没有经验的开发者来说也很容易使用。这种方法论最好的部分是，它不需要开发者对系统中的微架构和性能监控计数器（PMCs）有深入的理解，仍然能够有效地找到 CPU 瓶颈。

在概念层面上，TMA 识别了什么导致了程序执行的停滞。图 38 展示了 TMA 核心思想的示意图。这并不是实际分析的方式，因为分析每一个微操作（μop）将会非常缓慢。尽管如此，这个图表有助于理解方法论。

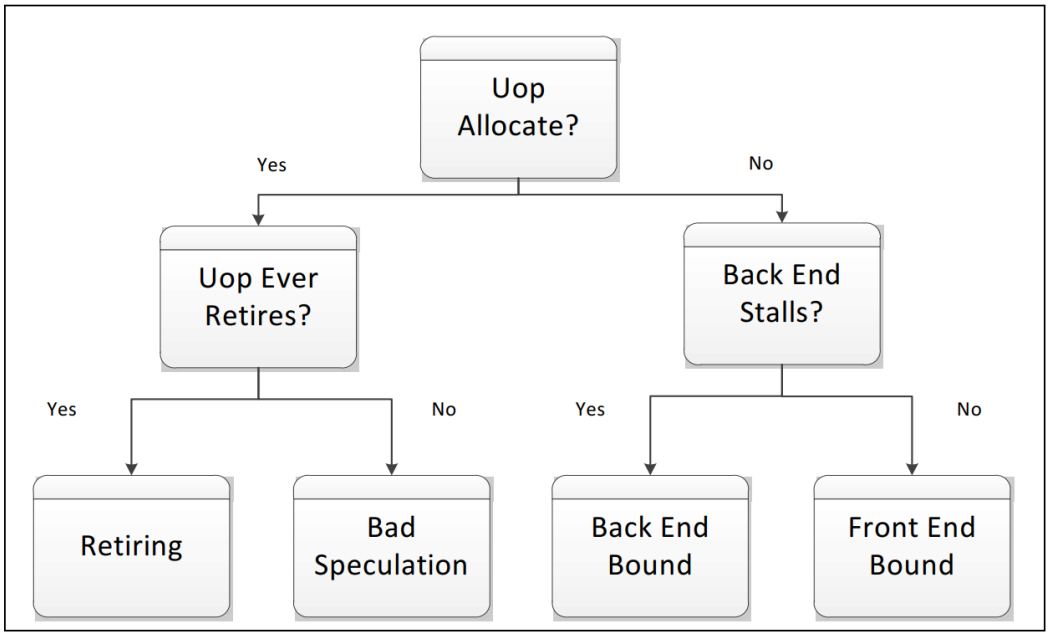


Figure 38: 自顶向下微架构分析顶层分解背后的概念。© 图片来自 [Yasin, 2014]

以下是如何阅读这个图表的简短指南。正如我们从 Chapter 3 所知，CPU 内部有缓冲区，用于跟踪正在执行的 μops 的信息。每当新的指令被获取和解码时，这些缓冲区中就会分配新的条目。如果在某个执行周期中，指令的 μop 没有被分配，可能是因为两个原因之一：要么我们无法获取和解码它（前端瓶颈(`Front End Bound`)）；要么后端工作过载，无法为新的 μop 分配资源（后端瓶颈(`Back End Bound`)）。如果一个 μop 被分配并安排执行但从未退役，这意味着它来自一个错误预测的路径（错误推测(`Bad Speculation`)）。最后，退役(`Retiring`)代表正常执行。这是我们希望所有 μops 所在的桶，尽管稍后我们会讨论一些例外情况。

为了实现其目标，TMA 通过监控一组特定的性能事件来观察程序的执行，然后根据预定义的公式计算指标。基于这些指标，TMA 通过将其分配到四个高级类别之一来对程序进行特征化。这四个高级类别每个都有若干嵌套级别，CPU 厂商可以选择不同的方式来实现。每一代处理器可能有不同的公式来计算这些指标，所以最好依赖工具来进行分析，而不是试图自己计算。

在接下来的章节中，我们将讨论 AMD、ARM 和 Intel 处理器中 TMA 的实现。

6.1.1 在英特尔平台上的 TMA

TMA 方法学首次由英特尔于 2014 年提出，并从 SandyBridge 系列处理器开始提供支持。英特尔的实现支持每个高级别桶的嵌套类别，从而更好地了解程序中的 CPU 性能瓶颈（参见图 39 ）。

该工作流程旨在“深入挖掘 (drill down)”TMA 层次结构的较低级别，直到我们达到对性能瓶颈的非常具体的分类为止。例如，首先，我们收集主要的四个桶的指标：`Front End Bound`、`Back End Bound`、`Retiring`、`Bad Speculation`。比如，我们发现程序执行的大部分时间被内存访问阻塞了（这是`Back End Bound`桶，参见图 39 ）。接下来的步骤是

再次运行工作负载，并仅收集与Memory Bound桶有关的特定指标。这个过程重复进行，直到我们知道确切的根本原因，例如，L3 Bound。

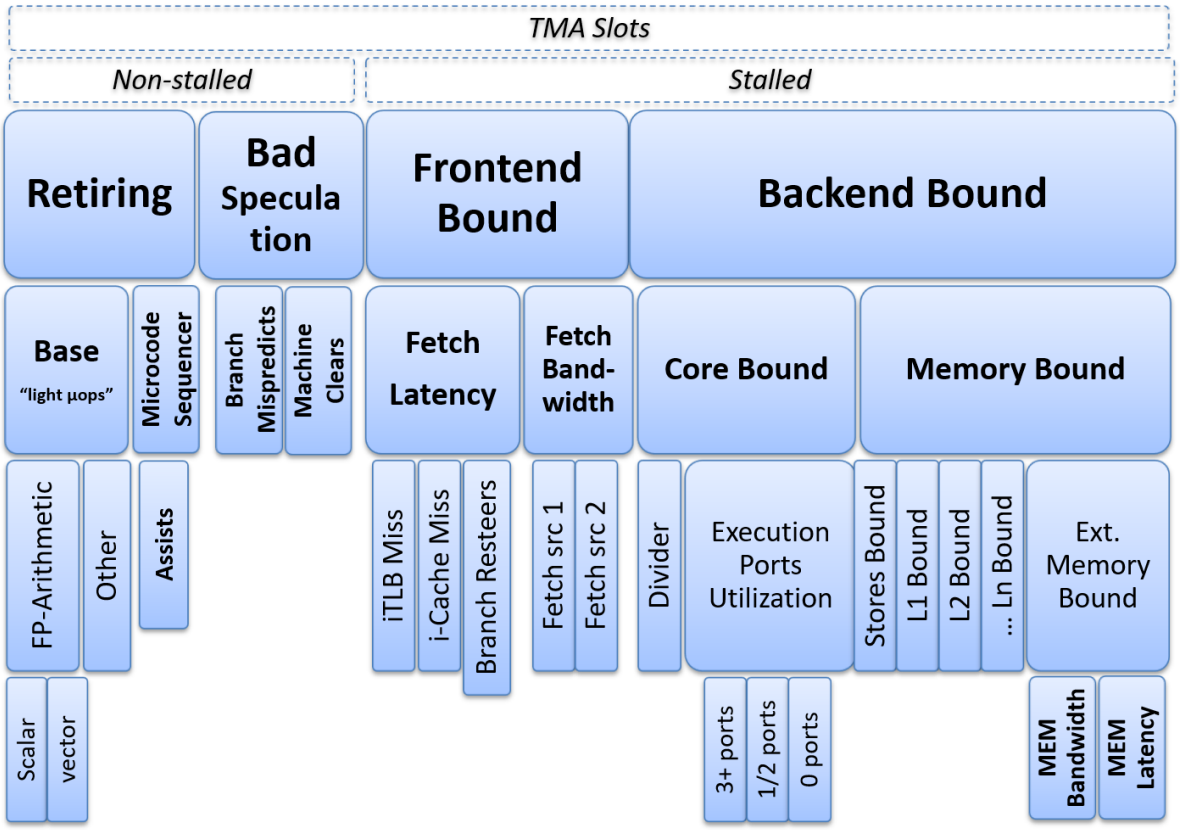


Figure 39: TMA 性能瓶颈的层次结构。© Image by Ahmad Yasin.

多次运行工作负载并在每次运行时专注于特定指标是完全可以的。但通常，运行一次工作负载并收集所有 TMA 各级别所需的所有指标就足够了。性能分析工具通过在单次运行中在不同性能事件之间进行多路复用（参见 Section 5.3.3）来实现这一点。此外，在现实应用中，性能可能受到多个因素的限制。例如，它可以同时经历大量的分支错误预测（Bad Speculation）和缓存丢失（Back End Bound）。在这种情况下，TMA 将同时深入挖掘多个桶，并确定每种类型的瓶颈对程序性能的影响。像英特尔的 VTune Profiler、AMD 的 uProf 和 Linux 的perf等分析工具可以在单次基准测试运行中计算所有 TMA 指标。但是，这仅在工作负载稳定时才可行。否则，最好回到多次运行和每次运行都进行深入挖掘的原始策略。

TMA 的前两个级别的指标以所有流水线插槽的百分比表示（参见 Section 4.5），这些插槽在程序执行期间是可用的。这使得 TMA 能够准确表示 CPU 微体系结构的利用率，考虑到处理器的全部带宽。到目前为止，所有内容都应该很好地总结为 100%。然而，从第 3 级开始，桶可能以不同的计数域表示，例如时钟和停顿。因此，它们不一定与其他 TMA 桶直接可比。

TMA 的第一步是识别程序中的性能瓶颈。在完成这一步之后，我们需要知道问题具体出现在代码的哪里。TMA 的第二步是将问题的源头定位到代码的确切行和汇编指令。该分析方法提供了应针对性能问题的每个类别使用的确切性能事件。然后，您可以在此事件上进行采样，以找到在第一阶段识别的性能瓶颈所在的源代码行。如果这个过程让您感到困惑，不用担心，阅读案例研究后一切都会变得清晰。

案例研究：使用 TMA 减少缓存未命中数量

作为本案例研究的示例，我们采用了非常简单的基准测试，使其易于理解和更改。它显然不能代表现实世界的应用程序，但足以演示 TMA 的工作流程。本书的第二部分中有更多实用的例子。

本书的大部分读者可能会将 TMA 应用于他们熟悉自己的应用程序。但即使您是第一次看到该应用程序，TMA 也非常有效。因此，我们不会首先向您展示基准测试的原始源代码。但这里有一个简短的描述：基准测试在堆上分配了一个 200 MB 的数组，然后进入一个 100M 次迭代的循环。在循环的每次迭代中，它都会生成一个指向已分配数组的随机索引，执行一些虚拟工作，然后从该索引读取值。

我们使用配备有 Intel Core i5-8259U CPU（基于 Skylake）和 16GB DRAM（DDR4 2400 MT/s）的机器运行实验，运行 64 位 Ubuntu 20.04（内核版本 5.13.0-27）。

步骤 1：识别瓶颈

作为第一步，我们运行微基准测试并收集一组有限的事件，这些事件将帮助我们计算第 1 级指标。在这里，我们尝试通过将它们归因于四个 L1 桶（“前端受限 (FE bound)”、“后端受限 (BE bound)”、“退役 (retiring)”、“错误推测 (Bad Speculation)”）来识别应用程序的高级性能瓶颈。可以使用 Linux `perf` 工具收集第 1 级指标。从 Linux 内核 4.8 开始，`perf` 在 `perf stat` 命令中有一个 `--topdown` 选项，用于打印 TMA 第 1 级指标。以下是我们基准测试的细分。本部分的命令输出经过修剪以节省空间。

[TODO]: 在 AlderLake 上, `perf stat --topdown`无法在内核 4.8 上工作，需要更新版本。现在它可以打印 L1 和 L2 TMA 指标。（请参阅 https://github.com/dendibakh/perf-book/issues/42）

```
$ perf stat --topdown -a -- taskset -c 0 ./benchmark.exe
      retiring  bad speculat  FE bound  BE bound
S0-C0    32.5%        0.2%       13.8%      53.4%  <==
S0-C1    17.4%        2.3%       12.0%      68.2%
S0-C2    10.1%        5.8%       32.5%      51.6%
S0-C3    47.3%        0.3%        2.9%      49.6%
...
```

为了获得高级 TMA 指标的值，Linux `perf` 需要分析整个系统 (`-a`)。这就是为什么我们看到所有内核的指标。但是由于我们已经使用 `taskset -c 0` 将基准测试固定在核心 0 上，因此我们只需要关注与 `S0-C0` 对应的行。我们可以丢弃其他行，因为它们正在运行其他任务或处于空闲状态。通过查看输出，我们可以判断应用程序的性能受 CPU 后端限制。现在先不进行分析，让我们向下钻取一层。

Linux perf 只支持 1 级 TMA 指标，因此要访问 2、3 级及更高级别的 TMA 指标，我们将使用 `toplev` 工具，它是 Andi Kleen 编写的 pmu-tools: https://github.com/andikleen/pmu-tools[87] 的一部分。它用 Python 实现，并在幕后调用 Linux `perf`。要使用 `toplev`，必须启用特定的 Linux 内核设置，有关详细信息，请查看文档。

```
$ ~/pmu-tools/toplev.py --core S0-C0 -l2 -v --no-desc taskset -c 0 ./benchmark.exe
...
# Level 1
S0-C0  Frontend_Bound:                13.92 % Slots
S0-C0  Bad_Speculation:                0.23 % Slots
S0-C0  Backend_Bound:                 53.39 % Slots
S0-C0  Retiring:                      32.49 % Slots
# Level 2
```

[87] PMU 工具 - https://github.com/andikleen/pmu-tools.

```
S0-C0  Frontend_Bound.FE_Latency:     12.11 % Slots
S0-C0  Frontend_Bound.FE_Bandwidth:    1.84 % Slots
S0-C0  Bad_Speculation.Branch_Mispred: 0.22 % Slots
S0-C0  Bad_Speculation.Machine_Clears: 0.01 % Slots
S0-C0  Backend_Bound.Memory_Bound:    44.59 % Slots <==
S0-C0  Backend_Bound.Core_Bound:       8.80 % Slots
S0-C0  Retiring.Base:                 24.83 % Slots
S0-C0  Retiring.Microcode_Sequencer:   7.65 % Slots
```

在此命令中，我们还将进程固定到 CPU0（使用 `taskset -c 0`），并将 `toplev` 的输出仅限于此核心（`--core S0-C0`）。选项 `-l2` 告诉工具收集 Level 2 指标。选项 `--no-desc` 禁用每个指标的描述。

我们可以看到，应用程序的性能受内存访问限制（`Backend_Bound.Memory_Bound`）。近一半的 CPU 执行资源都浪费在等待内存请求完成上。现在让我们更深入地挖掘一次[88]：

```
$ ~/pmu-tools/toplev.py --core S0-C0 -l3 -v --no-desc taskset -c 0 ./benchmark.exe
...
# Level 1
S0-C0    Frontend_Bound:                 13.91 % Slots
S0-C0    Bad_Speculation:                 0.24 % Slots
S0-C0    Backend_Bound:                  53.36 % Slots
S0-C0    Retiring:                       32.41 % Slots
# Level 2
S0-C0    FE_Bound.FE_Latency:            12.10 % Slots
S0-C0    FE_Bound.FE_Bandwidth:           1.85 % Slots
S0-C0    BE_Bound.Memory_Bound:          44.58 % Slots
S0-C0    BE_Bound.Core_Bound:             8.78 % Slots
# Level 3
S0-C0-T0 BE_Bound.Mem_Bound.L1_Bound:     4.39 % Stalls
S0-C0-T0 BE_Bound.Mem_Bound.L2_Bound:     2.42 % Stalls
S0-C0-T0 BE_Bound.Mem_Bound.L3_Bound:     5.75 % Stalls
S0-C0-T0 BE_Bound.Mem_Bound.DRAM_Bound:  47.11 % Stalls <==
S0-C0-T0 BE_Bound.Mem_Bound.Store_Bound:  0.69 % Stalls
S0-C0-T0 BE_Bound.Core_Bound.Divider:     8.56 % Clocks
S0-C0-T0 BE_Bound.Core_Bound.Ports_Util: 11.31 % Clocks
```

我们发现瓶颈在于 `DRAM_Bound`。这告诉我们，许多内存访问都会错过所有级别的缓存，并一直到达主内存。如果我们收集程序的 L3 缓存未命中绝对数量，也可以确认这一点。对于 Skylake 架构，`DRAM_Bound` 指标是使用 `CYCLE_ACTIVITY.STALLS_L3_MISS` 性能事件计算的。让我们手动收集它：

```
$ perf stat -e cycles,cycle_activity.stalls_l3_miss -- ./benchmark.exe
  32226253316  cycles
  19764641315  cycle_activity.stalls_l3_miss
```

`CYCLE_ACTIVITY.STALLS_L3_MISS` 事件会计算执行停顿时的周期数，而 L3 缓存未命中需求加载尚未完成。我们可以看到大约有 60% 的此类周期，这非常糟糕。

[88] 由于我们知道应用程序受内存限制，因此我们可以改用 `-l2 --nodes L1_Bound,L2_Bound,L3_Bound,DRAM_Bound,Store_Bound` 选项而不是 `-l3` 来限制收集。

步骤 2：定位代码中的位置

TMA 过程的第二个步骤是找到性能事件最频繁发生的代码位置。为此，应该使用与步骤 1 中确定的瓶颈类型相对应的事件对工作负载进行采样。

查找此类事件的推荐方法是使用 `toplev` 工具的 `--show-sample` 选项，该选项将建议可用于定位问题的 `perf record` 命令行。为了理解 TMA 的机制，我们还介绍了手动查找与特定性能瓶颈关联的事件的方法。性能瓶颈和用于确定瓶颈在源代码中的位置的性能事件之间的对应关系可以使用 TMA metrics: https://github.com/intel/perfmon/blob/main/TMA_Metrics.xlsx[89] 表格来完成。`Locate-with` 列表示用于定位问题发生确切代码位置的性能事件。在我们的例子中，为了找到导致 `DRAM_Bound` 指标如此高的内存访问（L3 缓存未命中），我们应该对 `MEM_LOAD_RETIRED.L3_MISS_PS` 精确事件进行采样。以下是示例命令：

```
$ perf record -e cpu/event=0xd1,umask=0x20,name=MEM_LOAD_RETIRED.L3_MISS/ppp ./benchmark.exe

$ perf report -n --stdio
...
# Samples: 33K of event 'MEM_LOAD_RETIRED.L3_MISS'
# Event count (approx.): 71363893
# Overhead   Samples  Shared Object   Symbol
# ........  ......... ..............  ..................
#
    99.95%    33811   benchmark.exe   [.] foo
     0.03%       52   [kernel]        [k] get_page_from_freelist
     0.01%        3   [kernel]        [k] free_pages_prepare
     0.00%        1   [kernel]        [k] free_pcppages_bulk
```

几乎所有 L3 未命中都是由可执行文件 `benchmark.exe` 中的函数 `foo` 中的内存访问引起的。现在是时候查看基准测试的源代码了，可以在 Github: https://github.com/dendibakh/dendibakh.github.io/tree/master/_posts/code/TMAM 上找到。[90]

为了避免编译器优化，函数 `foo` 是用汇编语言实现的，如 Listing 91 所示。基准测试的“驱动”部分在 `main` 函数中实现，如 Listing 1 所示。我们分配了一个足够大的数组 `a` 以使其不适合 6MB 的 L3 缓存。基准测试生成一个指向数组 `a` 的随机索引，并将此索引与数组 `a` 的地址一起传递给 `foo` 函数。稍后，`foo` 函数会读取此随机内存位置。[91]

清单：函数 foo 的汇编代码。

```
$ perf annotate --stdio -M intel foo
Percent |  Disassembly of benchmark.exe for MEM_LOAD_RETIRED.L3_MISS
------------------------------------------------------------
        :  Disassembly of section .text:
        :
        :  0000000000400a00 <foo>:
        :  foo():
   0.00 :    400a00:  nop  DWORD PTR [rax+rax*1+0x0]
   0.00 :    400a08:  nop  DWORD PTR [rax+rax*1+0x0]
                ...
```

[89] TMA 指标 - https://github.com/intel/perfmon/blob/main/TMA_Metrics.xlsx.

[90] 案例研究示例 - https://github.com/dendibakh/dendibakh.github.io/tree/master/_posts/code/TMAM.

[91] 根据 x86 调用约定 (https://en.wikipedia.org/wiki/X86_calling_conventions), 前两个参数分别位于 `rdi` 和 `rsi` 寄存器中。

```
100.00 :    400e07:  mov  rax,QWORD PTR [rdi+rsi*1] <==
                ...
  0.00 :    400e13:  xor  rax,rax
  0.00 :    400e16:  ret
```

Listing 1 Source code of function main.

```
extern "C" { void foo(char* a, int n); }
const int _200MB = 1024*1024*200;
int main() {
  char* a = (char*)malloc(_200MB); // 200 MB buffer
  ...
  for (int i = 0; i < 100000000; i++) {
    int random_int = distribution(generator);
    foo(a, random_int);
  }
  ...
}
```

通过查看 Listing 91，我们可以看到函数 foo 中的所有 L3 缓存未命中都被标记为单个指令。现在我们知道是哪条指令导致了这么多 L3 未命中，让我们来修复它。

步骤 3：修复问题

请记住，在 foo 函数的开头有用 NOP 模拟的虚拟工作。这会在我们获得将要访问的下一个地址的那一刻与实际加载指令之间创建一个时间窗口。这个时间窗口的存在使我们有机会在虚拟工作的同时开始预取内存位置。Listing 91 展示了这个想法。有关显式内存预取技术的更多信息，请参阅 Section 8.2。

清单：在 main 中插入内存预取。

```
  for (int i = 0; i < 100000000; i++) {
    int random_int = distribution(generator);
+   __builtin_prefetch ( a + random_int, 0, 1);
    foo(a, random_int);
  }
```

通过这个显式的内存预取提示，执行时间从 8.5 秒减少到 6.5 秒。此外，CYCLE_ACTIVITY.STALLS_L3_MISS 事件的数量几乎减少了十倍：从 19B 减少到 2B。

TMA 是一个迭代过程，因此一旦我们解决了一个问题，就需要从第一步开始重复这个过程。很可能瓶颈会转移到另一个类别，在这个例子中是Retiring。这是一个简单的例子，展示了 TMA 方法论的工作流程。分析现实世界的应用程序不太可能那么简单。书的第二部分章节的组织旨在方便与 TMA 过程一起使用。特别是，第 8 章涵盖了Memory Bound类别，第 9 章涵盖了Core Bound，第 10 章涵盖了Bad Speculation，第 11 章涵盖了FrontEnd Bound。这样的结构意图是形成一个清单，当你遇到特定的性能瓶颈时，可以用来驱动代码变更。

其他资源和链接

- Ahmad Yasin 的论文“用于性能分析和计数器架构的顶向下方法”[Yasin, 2014]。
- Ahmad Yasin 在 IDF'15 上的演讲“使用英特尔 Skylake 的顶向下分析使软件优化变得简单”，网址：https://youtu.be/kjufVhyuV_A。
- Andi Kleen 的博客：pmu-tools，第二部分：toplev，网址：http://halobates.de/blog/p/262。

- Toplev 手册，网址：https://github.com/andikleen/pmu-tools/wiki/toplev-manual。

6.1.2 TMA 在 AMD 平台上

从 Zen4 开始，AMD 处理器支持一级和二级 TMA 分析。根据 AMD 文档，它被称为“流水线利用率”分析，但基本思想保持不变。L1 和 L2 桶也与 Intel 的非常相似。Linux 用户可以使用 `perf` 工具收集流水线利用率数据。

接下来，我们将研究 Crypto++: https://github.com/weidai11/cryptopp[92] 实现的 SHA-256（安全散列算法 256），它是比特币挖掘的基本密码算法。Crypto++ 是一个开源的 C++ 密码算法类库，包含许多算法的实现，不仅仅是 SHA-256。但是，对于我们的示例，我们通过注释掉 `bench1.cpp` 中 `BenchmarkUnkeyedAlgorithms` 函数中相应的行来禁用所有其他算法的基准测试。

我们在配备 Ubuntu 22.04、Linux 内核 6.5.0-15-generic 的 AMD Ryzen 9 7950X 机器上运行了测试。我们使用 GCC 12.3 C++ 编译器编译了 Crypto++ 版本 8.9。我们使用了默认的 `-O3` 优化选项，但由于代码是用 x86 内在函数编写的（请参阅 Section 9.4）并利用了 SHA x86 ISA 扩展，因此对性能影响不大。

下面是我们用来获取 L1 和 L2 流水线利用率指标的命令。输出经过修剪，删除了一些统计数据以消除不必要的干扰。

```
$ perf stat -M PipelineL1,PipelineL2 -- ./cryptest.exe b1 10
 0.0 %  bad_speculation_mispredicts        (20.08%)
 0.0 %  bad_speculation_pipeline_restarts  (20.08%)
 0.0 %  bad_speculation                    (20.08%)
 6.1 %  frontend_bound                     (20.00%)
 6.1 %  frontend_bound_bandwidth           (20.00%)
 0.1 %  frontend_bound_latency             (20.00%)
65.9 %  backend_bound_cpu                  (20.00%)
 1.7 %  backend_bound_memory               (20.00%)
67.5 %  backend_bound                      (20.00%)
26.3 %  retiring                           (20.08%)
20.2 %  retiring_fastpath                  (19.99%)
 6.1 %  retiring_microcode                 (19.99%)
```

在输出中，括号中的数字表示运行时持续时间的百分比，当时正在监控指标。正如我们看到的，由于多路复用，所有指标只被监控了 20% 的时间。在我们的案例中，这可能不是问题，因为 SHA256 具有一致的行为，但并非总是如此。为了最小化多路复用的影响，您可以在单个运行中收集一组有限的指标，例如 `perf stat -M frontend_bound,backend_bound`。

上面显示的流水线利用率指标的描述可以在 [AMD, 2024, 第 2.8 章 流水线利用率] 中找到。通过查看这些指标，我们可以看到分支预测不会在 SHA256 中发生（`bad_speculation` 为 0%）。仅使用了可用调度槽位的 26.3%（`retiring`），这意味着其余 73.7% 由于前端和后端停顿而浪费。

加密指令并非简单，因此在内部被分解成更小的片段（µops）。一旦处理器遇到这样的指令，它就会从微码中检索它的 µops。微操作是从微码排序器获取的，带宽低于常规指令解码器，使其成为性能瓶颈的潜在来源。Crypto++ SHA256 实现大量使用诸如 `SHA256MSG2`, `SHA256RNDS2` 等指令，这些指令根据 uops.info: https://uops.info/table.html[93] 网站由多个 µops 组成。`retiring_microcode` 指标表明 6.1% 的调度槽位被微码操作使用。由于前端的带宽瓶颈，相同数量的调度槽位未使用（`frontend_bound_bandwidth`）。这两个指标共同表明，这 6.1% 的调度槽位被浪费，因为微码排序器没有提供 µops，而后端本可以消耗它们。

[92] Crypto++ - https://github.com/weidai11/cryptopp

[93] uops.info - https://uops.info/table.html

大多数周期都停滞在 CPU 后端（backend_bound），但只有 1.7% 的周期由于等待内存访问而停滞（backend_bound_memory）。因此，我们知道基准测试主要受机器的计算能力限制。正如您将在本书第二部分中了解到的，这可能与数据流依赖性或某些加密操作的执行吞吐量有关。它们比传统的 ADD, SUB, CMP 等指令不那么频繁，因此通常只能在单个执行单元上执行。大量这样的操作可能会使该特定单元的执行吞吐量饱和。进一步的分析应该更仔细地观察源代码和生成的汇编代码，检查执行端口利用率，查找数据依赖性等；我们将在此停止。

就 Windows 而言，在撰写本文时，TMA 方法仅在服务器平台（代号 Genoa）上受支持，而不支持客户端系统（代号 Raphael）。TMA 支持在 AMD uProf 版本 4.1 中添加，但仅在命令行工具 AMDuProfPcm 工具中，它是 AMD uProf 安装的一部分。您可以参考 [AMD, 2024, 第 2.8 章 流水线利用率] 了解更多有关如何运行分析的详细信息。AMD uProf 的图形版本还没有 TMA 分析。

6.1.3 TMA 在 ARM 平台上

ARM CPU 架构设计者也为他们的处理器开发了一种 TMA 性能分析方法，我们接下来将讨论。ARM 在其文档中将其称为“Topdown”[Arm, 2023a]，因此我们将使用他们的命名。在撰写本章节时（2023 年底），Topdown 仅支持 ARM 设计的内核，例如 Neoverse N1 和 Neoverse V1 及其衍生产品，例如 Ampere Altra 和 AWS Graviton3。如果您需要刷新有关 ARM 芯片系列的记忆，请参考本书末尾的主要 CPU 微架构列表。Apple 设计的处理器目前还不支持 ARM Topdown 性能分析方法。

Neoverse V1 是 Neoverse 系列中第一个支持全套 1 级 Topdown 指标的 CPU：Bad Speculation、Frontend Bound、Backend Bound 和 Retiring。据说未来的 Neoverse 内核将支持更高级别的 TMA。在撰写本文时，没有针对 Neoverse N2 和 V2 内核的分析指南。在 V1 内核之前，Neoverse N1 只支持两个 L1 类别：Frontend Stalled Cycles 和 Backend Stalled Cycles。

为了演示基于 V1 处理器的 ARM Topdown 分析，我们启动了一个由 AWS Graviton3 驱动的 AWS EC2 m7g.metal 实例。请注意，由于虚拟化，Topdown 可能无法在其他非金属实例类型上运行。我们请求了由 AWS 管理的 64 位 ARM Ubuntu 22.04 LTS 以及 Linux kernel 6.2。提供的 m7g.metal 实例有 64 个 vCPU 和 256 GB 的内存。

我们将 Topdown 方法应用于 AI Benchmark Alpha: https://ai-benchmark.com/alpha.html，[94] 它是一个用于评估各种硬件平台（包括 CPU、GPU 和 TPU）的 AI 性能的开源 Python 库。该基准测试依赖 TensorFlow 机器学习库来测量关键深度学习模型的推理和训练速度。AI Benchmark Alpha 总共包含 42 个测试，包括分类、图像分割、文本翻译等等。

ARM 工程师开发了 topdown-tool: https://learn.arm.com/install-guides/topdown-tool/[95]，我们将在下面使用它。该工具可以在 Linux 和 Windows 上的 ARM 上运行。在 Linux 上，它使用标准的 perf 工具，而在 Windows 上，它使用 WindowsPerf: https://gitlab.com/Linaro/WindowsPerf/windowsperf[96]，这是一款 Windows on Arm 性能分析工具。类似于 Intel 的 TMA，ARM 的方法也采用了“向下钻取”的概念，即首先确定高级性能瓶颈，然后向下钻取更细致的根本原因分析。以下是我们使用的命令：

```
$ topdown-tool --all-cpus -m Topdown_L1 -- python -c "from ai_benchmark import AIBenchmark; results
    = AIBenchmark(use_CPU=True).run()"
Stage 1 (Topdown metrics)
=========================
[Topdown Level 1]
Frontend Bound... 16.48% slots
Backend Bound.... 54.92% slots
Retiring......... 27.99% slots
Bad Speculation..  0.59% slots
```

[94] AI Benchmark Alpha - https://ai-benchmark.com/alpha.html

[95] ARM topdown-tool - https://learn.arm.com/install-guides/topdown-tool/

[96] WindowsPerf - https://gitlab.com/Linaro/WindowsPerf/windowsperf

其中 --all-cpus 选项启用所有 CPU 的系统级收集，而 -m Topdown_L1 则收集 Topdown 1 级指标。-- 后面的所有内容都是运行 AI Benchmark Alpha 套件的命令行。

从上面的输出中，我们可以得出结论，基准测试不会出现分支预测错误。此外，如果不深入了解所涉及的工作负载，就很难说 16.5% 的“前端瓶颈”是否值得关注，因此我们将注意力转移到“后端瓶颈”指标上，该指标显然是停滞周期的主要来源。基于 Neoverse V1 的芯片没有二级细分，相反，该方法建议通过收集一组相应的指标来进一步探索有问题的类别。以下是我们如何深入研究更详细的“后端瓶颈”分析：

```
$ topdown-tool --all-cpus -n BackendBound -- python -c "from ai_benchmark import AIBenchmark;
    results = AIBenchmark(use_CPU=True).run()"
Stage 1 (Topdown metrics)
=========================
[Topdown Level 1]
Backend Bound..................... 54.70% slots

Stage 2 (uarch metrics)
=======================
  [Data TLB Effectiveness]
  DTLB MPKI......................... 0.413 misses per 1,000 instructions
  L1 Data TLB MPKI.................. 3.779 misses per 1,000 instructions
  L2 Unified TLB MPKI............... 0.407 misses per 1,000 instructions
  DTLB Walk Ratio................... 0.001 per TLB access
  L1 Data TLB Miss Ratio............ 0.013 per TLB access
  L2 Unified TLB Miss Ratio......... 0.112 per TLB access

  [L1 Data Cache Effectiveness]
  L1D Cache MPKI.................... 13.114 misses per 1,000 instructions
  L1D Cache Miss Ratio.............  0.046 per cache access

  [L2 Unified Cache Effectiveness]
  L2 Cache MPKI..................... 1.458 misses per 1,000 instructions
  L2 Cache Miss Ratio............... 0.027 per cache access

  [Last Level Cache Effectiveness]
  LL Cache Read MPKI................ 2.505 misses per 1,000 instructions
  LL Cache Read Miss Ratio.......... 0.219 per cache access
  LL Cache Read Hit Ratio........... 0.783 per cache access

  [Speculative Operation Mix]
  Load Operations Percentage........ 25.36% operations
  Store Operations Percentage.....  2.54% operations
  Integer Operations Percentage..... 29.60% operations
  Advanced SIMD Operations Percentage. 10.93% operations
  Floating Point Operations Percentage  6.85% operations
  Branch Operations Percentage...... 10.04% operations
  Crypto Operations Percentage.....  0.00% operations
```

Misc 类别包含不在主类别中的指令。例如，barriers

在上面的命令中，选项 `-n BackendBound` 收集与 `Backend Bound` 类别及其后代相关的所有指标。输出中每个指标的描述在 [Arm, 2023a] 中给出。请注意，它们与我们在 Section 4.11 中讨论的非常相似，因此您可能也想重新查看它。

我们的目标不是优化基准测试，而是要描述性能瓶颈。但是，如果有这样的任务，我们的分析可以继续进行。有大量的 `L1 Data TLB` 未命中（3.8 MPKI），但随后 90% 的未命中命中 L2 TLB（参见 `L2 Unified TLB Miss Ratio`）。总而言之，只有 0.1% 的 TLB 未命中导致页表遍历（参见 `DTLB Walk Ratio`），这表明这不是我们的主要关注点，尽管快速使用大页面的实验仍然值得。

查看 `L1/L2/LL Cache Effectiveness` 指标，我们可以发现数据缓存未命中的潜在问题。对 L1D 缓存的 ~22 次访问中就有一次会导致未命中（参见 `L1D Cache Miss Ratio`），这是可以容忍但仍然很昂贵的。对于 L2，这个数字是 37 分之一（参见 `L2 Cache Miss Ratio`），这要好得多。然而对于 LLC，`LL Cache Read Miss Ratio` 是不令人满意的：每 4 次访问就会导致一次失败。由于这是一个 AI 基准测试，其中大部分时间可能花在矩阵乘法上，因此循环阻塞等代码转换可能会有所帮助（参见 Section 9.3）。

最后一类给出了操作组合，这在某些情况下很有用。在我们的例子中，我们应该关注 SIMD 操作的低百分比，特别是考虑到使用了高度优化的 `Tensorflow` 和 `numpy` 库。相比之下，整数运算和分支的百分比似乎太高了。分支可能来自 Python 解释器或过多的函数调用。而高百分比的整数操作可能是由于缺乏向量化或线程同步造成的。[Arm, 2023a] 给出了一个使用来自 `Speculative Operation Mix` 类别的数据发现向量化机会的示例。

在我们的案例研究中，我们运行了两次基准测试，但在实践中，一次运行通常就足够了。运行没有选项的 `topdown-tool` 将使用单次运行收集所有可用的指标。此外，`-s combined` 选项将按 L1 类别对指标进行分组，并以类似于 `Intel Vtune`、`toplev` 和其他工具的格式输出数据。进行多次运行的唯一实际原因是工作负载具有突发行为，其非常短的阶段具有不同的性能特征。在这种情况下，您希望避免事件多路复用（参见 Section 5.3.3）并通过多次运行工作负载来提高收集准确性。

AI Benchmark Alpha 有各种可能表现出不同性能特征的测试。上面显示的输出汇总了所有基准并给出了总体细分。如果单个测试确实存在不同的性能瓶颈，这通常不是一个好主意。您需要对每个测试进行单独的 Topdown 分析。`topdown-tool` 可以提供帮助的一种方法是使用 `-i` 选项，该选项将根据可配置的时间间隔输出数据。然后，您可以比较间隔并决定下一步。

6.1.4 TMA 总结

TMA 非常适合识别 CPU 性能瓶颈。理想情况下，当我们在应用程序上运行它时，我们希望看到 “Retiring” 指标达到 100%。尽管存在例外。“Retiring” 指标达到 100% 意味着 CPU 已满负荷工作，并且以全速处理指令。但这并不能说明这些指令的质量。程序可以在紧密循环中等待锁，这将显示高 “Retiring” 指标，但不会做任何有用的工作。

另一个您可能看到高 “Retiring” 值但整体性能较慢的例子是程序存在未向量化的热点。您通过让处理器运行简单非向量化操作来让它 “轻松”，但这真的是利用可用 CPU 资源的最佳方式吗？当然不是。如果 CPU 没有执行代码的问题，并不意味着性能无法提高。注意这种情况，并记住 TMA 识别 CPU 性能瓶颈，但不会将其与程序性能相关联。您一旦进行必要的实验就会发现这一点。

虽然在玩具程序上实现 “Retiring” 接近 100% 是可能的，但现实世界的应用程序远不能达到。图 40 展示了 Google 数据中心工作负载的顶级 TMA 指标以及在 Intel 的 IvyBridge 服务器处理器上运行的几个 SPEC CPU2006: http://spec.org/cpu2006/[97] 基准测试。我们可以看到，大多数数据中心工作负载在 “Retiring” 桶中所占的比例非常小。这意味着大多数数据中心工作负载都会花时间停滞在各种瓶颈上。“BackendBound” 是性能问题的首要来源。“FrontendBound” 类别对于数据中心工作负载来说比 SPEC2006 更重要，因为这些应用程序通常具有庞大的代码库。最后，一些工作负载比其他工作负载更易受分支预测错误的影响，例如 “search2” 和 “445.gobmk”。

[97] SPEC CPU 2006 - http://spec.org/cpu2006/.

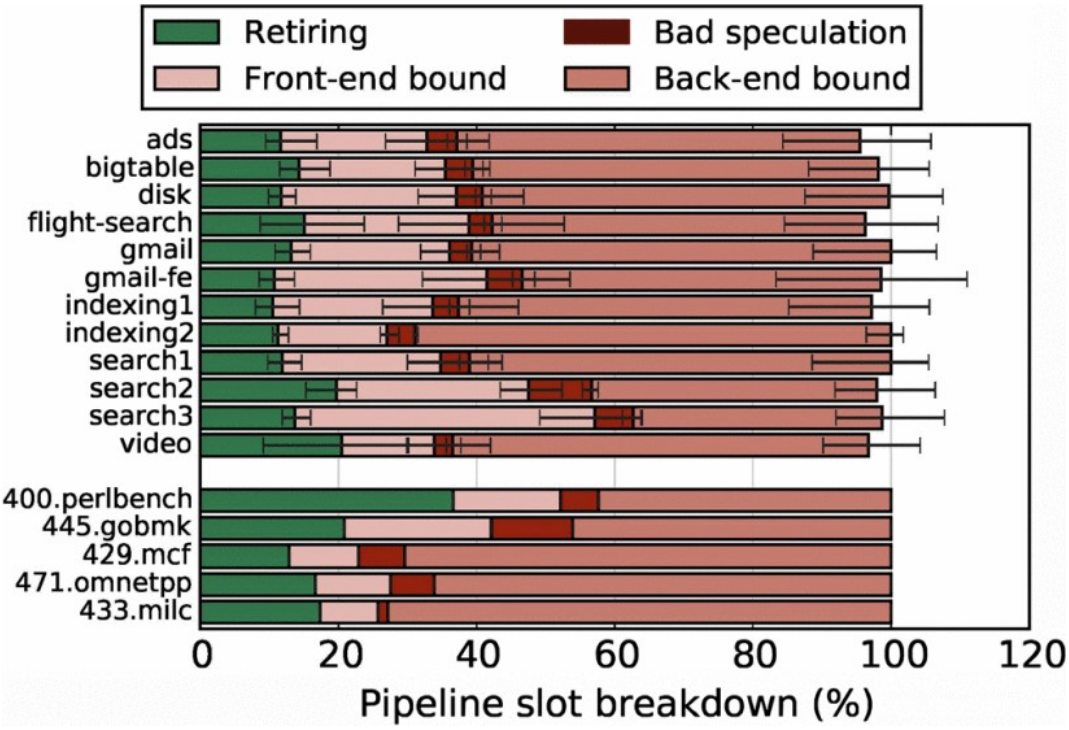


Figure 40: 谷歌数据中心工作负载的 TMA 分解以及几个 SPEC CPU2006 基准, © Image from [Kanev et al., 2015]

请记住，随着架构师不断尝试改进 CPU 设计，这些数字可能会随着其 CPU 而改变。这些数字也可能会随着其他指令集架构 (ISA) 和编译器版本的改变而改变。

在我们继续讨论之前，还有一些最后的思考…… 不建议在性能存在重大缺陷的代码上使用 TMA，因为它可能会将您引入错误的方向，并且您将修复真正的性能问题，而不是调整糟糕的代码，这只会浪费时间。同样，确保环境不会妨碍分析。例如，如果您丢弃文件系统缓存并在 TMA 下运行基准测试，它可能会显示您的应用程序受到内存限制，而实际上，当文件系统缓存预热时，这可能是错误的。

TMA 提供的工作负载特征描述可以将潜在优化的范围扩展到源代码之外。例如，如果应用程序受内存带宽限制，并且已经用尽所有可能的软件层面加速方法，那么可以通过升级内存子系统以使用更快的内存芯片来提高性能。这说明了如何使用 TMA 诊断性能瓶颈来支持您决定在新硬件上花钱。

6.2 分支记录机制（Branch Recording Mechanisms）

现代高性能 CPU 提供分支记录机制，使处理器能够连续记录一组先前执行的分支。但在进入细节之前，你可能会问：为什么我们对分支如此感兴趣？嗯，因为这是我们如何确定程序控制流的方式。我们基本上忽略基本块（参见 Section 11.2）中的其他指令，因为分支总是基本块中的最后一个指令。由于基本块中的所有指令都保证执行一次，因此我们只能关注将“代表”整个基本块的分支。因此，如果我们跟踪每个分支的结果，就可以重建程序的整个逐行执行路径。事实上，这就是英特尔处理器跟踪 (PT) 功能可以做到的，它在附录 D 中讨论。我们将在这里讨论的分支记录机制基于采样而不是跟踪，因此具有不同的用例和功能。

由英特尔、AMD 和 ARM 设计的处理器都宣布了他们的分支记录扩展。确切的实现可能会有所不同，但基本思想是相同的。硬件并行记录每个分支的“来自”和“到”地址以及一些额外数据，同时执行程序。如果我们收集足够长的源目的地对历史记录，我们将能够像调用堆栈一样解开程序的控制流，但深度有限。此类扩展旨在使正在运行的程序的运行速度降低到最小，通常在 1% 以内。

如果使用分支记录机制，我们可以在分支（或周期，没关系）上进行采样，但在每个采样期间，查看先前执行的 N

个分支。这使我们在热门代码路径中合理地覆盖了控制流，但不会让我们因为只检查了总数较少的分支而获得过多信息。请务必记住，这仍然是采样，因此并不是每个执行的分支都可以被检查。CPU 通常执行得太快，无法做到这一点。

非常重要的一点是，只有被执行的分支（taken branches）才会被记录。Listing 6.2展示了如何跟踪分支结果的一个例子。这段代码表示一个循环，其中包含三个可能改变程序执行路径的指令，即循环回边 (loop backedge)JNE（1）、条件分支JNS（2）、函数调用CALL（3），以及从这个函数返回（4，未显示）。

代码清单：记录分支的示例。

```
----> 4eda10:  mov   edi,DWORD PTR [rbx]
|     4eda12:  test  edi,edi
| --- 4eda14:  jns   4eda1e              <== (2)
| |   4eda16:  mov   eax,edi
| |   4eda18:  shl   eax,0x7
| |   4eda1b:  lea   edi,[rax+rdi*8]
|   > 4eda1e:  call  4edb26              <== (3)
|     4eda23:  add   rbx,0x4             <== (4)
|     4eda27:  mov   DWORD PTR [rbx-0x4],eax
|     4eda2a:  cmp   rbx,rbp
 ---- 4eda2d:  jne   4eda10              <== (1)
```

以下是使用分支记录机制可以记录的可能分支历史之一。它显示了执行 CALL 指令时最近的 7 个分支结果（未显示更多）。由于在循环的最新迭代中没有执行 JNS 分支 (4eda14 -> 4eda1e)，因此它没有被记录，因此不会出现在历史记录中。

```
    Source Address    Destination Address
    ...               ...
(1) 4eda2d            4eda10    <== next iteration
(2) 4eda14            4eda1e    <== jns taken
(3) 4eda1e            4edb26    <== call a function
(4) 4b01cd            4eda23    <== return from a function
(1) 4eda2d            4eda10    <== next iteration
(3) 4eda1e            4edb26    <== latest branch
```

6.3 未记录未执行的分支

未记录未执行的分支可能会增加分析负担，但通常不会使其过于复杂。由于我们知道控制流从条目 N-1 的目标地址到条目 N 的源地址是顺序的，因此我们仍然可以推断完整的执行路径。

接下来，我们将分别看一下每个供应商的分支记录机制，然后探讨如何在性能分析中使用它们。

6.3.1 英特尔平台上的 LBR

英特尔首次在其 Netburst 微架构中实现了其最后分支记录 (LBR) 功能。最初，它只能记录最近的 4 个分支结果。从 Nehalem 开始增加到 16，从 Skylake 开始增加到 32。在 Goldencove 微架构之前，LBR 被作为一组特定于模型的寄存器 (MSR) 实现，但现在它在架构寄存器内工作。其主要优点是 LBR 功能清晰可见，无需检查当前 CPU 的确切型号。这使操作系统和分析工具中的支持更加容易。此外，LBR 条目可以配置为包含在 PEBS 记录中（参见 Section 6.5）。

LBR 寄存器的作用类似于一个环形缓冲区，它不断地被覆盖，并提供最近 32 个分支结果。每个 LBR 条目由三个 64 位值组成：

- 分支的源地址（From IP）。
- 分支的目标地址（To IP）。
- 操作的元数据，包括误预测和经过的周期时间信息。

除了源地址和目标地址之外，保存的额外信息也有重要的应用，我们将在后面讨论。

当采样计数器溢出并触发性能监控中断（PMI）时，LBR 记录会冻结，直到软件捕获 LBR 记录并恢复收集。

LBR 收集可以限制在一组特定的分支类型上，例如，用户可以选择只记录函数调用和返回。在对 Listing 6.2中的代码应用这样的过滤器时，我们只会在历史记录中看到分支（3）和（4）。用户还可以过滤进出条件和无条件跳转、间接跳转和调用、系统调用、中断等。在 Linux perf 中有一个-j选项，用于启用/禁用记录各种分支类型。

默认情况下，LBR 数组作为一个环形缓冲区，捕获控制流转换。然而，LBR 数组的深度是有限的，这可能是在分析某些应用程序时的一个限制因素，在这些应用程序中，执行流的转换伴随着大量的叶子函数调用。这些对叶子函数的调用及其返回，很可能会将主执行上下文从 LBR 中移出。再次考虑 Listing 6.2中的例子。假设我们想从 LBR 历史记录中展开调用栈，并且我们配置 LBR 只捕获函数调用和返回。如果循环运行数千次，考虑到 LBR 数组只有 32 个条目深，我们很可能只会看到 16 对条目（3）和（4）。在这种情况下，LBR 数组充满了叶子函数调用，这些调用无助于我们展开当前的调用栈。

这就是为什么 LBR 支持调用栈模式。启用此模式后，LBR 数组像以前一样捕获函数调用，但当执行返回指令时，最后捕获的分支（call）记录会以后进先出（LIFO）的方式从数组中清除。因此，与已完成的叶子函数相关的分支信息不会被保留，同时保留了主线执行路径的调用栈信息。当以这种方式配置时，LBR 数组模拟了一个调用栈，其中CALL被“推入”（pushed），RET被“弹出”（popped）条目。如果你的应用程序中的调用栈深度从未超过 32 个嵌套帧，LBRs 将为你提供非常准确的信息。[Intel, 2023b, Volume 3B, Chapter 19 Last Branch Records]

可以使用以下命令确保你的系统上的 LBR 已启用：

```
$ dmesg | grep -i lbr
[    0.228149] Performance Events: PEBS fmt3+, 32-deep LBR, Skylake events, full-width counters,
    Intel PMU driver.
```

使用 Linux 的perf，可以使用以下命令收集 LBR 堆栈：

```
$ perf record -b -e cycles ./benchmark.exe
[ perf record: Woken up 68 times to write data ]
[ perf record: Captured and wrote 17.205 MB perf.data (22089 samples) ]
```

LBR 堆栈也可以使用 perf record --call-graph lbr 命令收集，但是收集的信息量少于使用 perf record -b。例如，在运行 perf record --call-graph lbr 时不会收集分支预测和周期数据。

因为每个收集的样本都捕获整个 LBR 堆栈（32 个最后的分支记录），所以收集的数据（perf.data）的大小比不使用 LBR 的采样要大得多。尽管如此，在大多数 LBR 使用案例中，运行时开销低于 1%。[Nowak & Bitzes, 2014]

用户可以导出原始 LBR 堆栈进行自定义分析。以下是可以用来转储收集的分支堆栈内容的 Linux perf 命令：

```
$ perf record -b -e cycles ./benchmark.exe
$ perf script -F brstack &> dump.txt
```

dump.txt文件可能非常大，包含如下所示的行：

```
...
0x4edaf9/0x4edab0/P/-/-/29
0x4edabd/0x4edad0/P/-/-/2
0x4edadd/0x4edb00/M/-/-/4
0x4edb24/0x4edab0/P/-/-/24
0x4edabd/0x4edad0/P/-/-/2
0x4edadd/0x4edb00/M/-/-/1
0x4edb24/0x4edab0/P/-/-/3
0x4edabd/0x4edad0/P/-/-/1
...
```

上述输出展示了 LBR 堆栈中的 8 个条目，LBR 堆栈通常包含 32 个条目。每个条目都有 `FROM` 和 `TO` 地址（十六进制值）、预测标志 (`M` - 预测错误，`P` - 预测正确）以及周期数（每个条目最后一个位置的数字）。用 "`-`" 标记的组件与事务内存扩展 (TSX) 相关，我们将在本文中不进行讨论。好奇的读者可以参考 `perf script` 规范: http://man7.org/linux/man-pages/man1/perf-script.1.html[98] 中解码的 LBR 条目的格式。

6.3.2 AMD 平台上的 LBR

AMD 处理器也在 AMD Zen4 处理器上支持最后分支记录（LBR）。Zen4 有 16 对 "from" 和 "to" 地址记录以及一些额外的元数据。与 Intel LBR 类似，AMD 处理器有能力记录各种类型的分支。与 Intel LBR 的主要区别在于，AMD 处理器目前还不支持调用栈模式，因此 LBR 特性不能用于调用栈收集。另一个值得注意的区别是 AMD LBR 记录中没有周期计数字段。更多详细信息请参见 [AMD, 2023, 13.1.1.9 Last Branch Stack Registers]。

从 Linux 内核 6.1 开始，除非另有明确说明，Linux 'perf' 在 AMD Zen4 处理器上支持我们下面讨论的分支分析用例。收集 AMD LBRs 的 Linux `perf`命令使用相同的`-b`和`-j`选项。

使用 AMD uProf CLI 工具也可以进行分支分析。以下示例命令将转储收集的原始 LBR 记录并生成 CSV 报告:

```
$ AMDuProfCLI collect --branch-filter -o /tmp/ ./AMDTClassicMatMul-bin
```

6.4 ARM 平台上的 BRBE

ARM 在 2020 年作为 ARMv9.2-A ISA 的一部分推出了其名为 BRBE 的分支记录扩展。ARM BRBE 与英特尔的 LBR 非常相似，提供了许多类似的功能。就像英特尔的 LBR 一样，BRBE 记录也包含源地址和目标地址、预测错误位和周期计数值。根据最新可用的 BRBE 规范，不支持调用堆栈模式。分支记录仅包含已在架构上执行的分支的信息，即不在预测错误路径上。用户还可以根据特定分支类型过滤记录。一个值得注意的区别是 BRBE 支持可配置的 BRBE 缓冲区深度：处理器可以选择 BRBE 缓冲区的容量为 8、16、32 或 64 个记录。更多细节可在 [Arm, 2022a, 章节 F1 "Branch Record Buffer Extension"] 中找到。

在撰写本文时，还没有商用机器实现 ARMv9.2-A，因此无法测试此扩展的实际运行情况。

6.4.1 捕获调用堆栈

分支记录使许多重要用例成为可能。在本节和接下来的几节中，我们将介绍最重要的几个用例。

分支记录最流行的用例之一是捕获调用堆栈。我们已经在 Section 5.5.3 中介绍了为什么需要收集它们。即使你编译了一个没有帧指针或调试信息的程序，分支记录也可以用作收集调用图信息的轻量级替代方法。

在撰写本文时 (2023 年)，AMD 的 LBR 和 ARM 的 BRBE 不支持调用堆栈收集，但英特尔的 LBR 支持。以下是你可以使用英特尔 LBR 执行此操作的方法:

[98] Linux `perf script` 手册页 - http://man7.org/linux/man-pages/man1/perf-script.1.html.

```
$ perf record --call-graph lbr -- ./a.exe
$ perf report -n --stdio
# Children   Self    Samples  Command  Object  Symbol
# ........  .......  .......  .......  ......  .......
  99.96%  99.94%    65447    a.exe    a.exe  [.] bar
          |
           --99.94%--main
                     |
                     |--90.86%--foo
                     |          |
                     |           --90.86%--bar
                     |
                      --9.08%--zoo
                                bar
```

正如你所见，我们已经确定了程序中最热的功能（即 bar）。我们还发现调用者对函数 bar 中花费的大部分时间做出了贡献：该工具捕获了 main->foo->bar 调用堆栈 91% 的时间，捕获了 main->zoo->bar 9% 的时间。换句话说，bar 中 91% 的样本都将 foo 作为其调用者函数。

值得一提的是，在这种情况下，我们不一定能得出关于函数调用次数的结论。例如，我们不能说 foo 调用 bar 的频率比 zoo 高 10 倍。可能的情况是，foo 调用 bar 一次，但在 bar 内部执行了昂贵的路径，而 zoo 调用 bar 多次，但很快就返回。

6.4.2 识别热点分支

分支记录还使我们能够知道哪些分支被采取的频率最高。它在 Intel 和 AMD 上都支持。根据 ARM 的 BRBE 规范，它可以支持，但由于缺乏实现此扩展的处理器，无法验证。这里是一个例子：

TODO：检查："添加 -F +srcline_from,srcline_to 会降低构建报告的速度。希望在更高版本的 perf 中，解码时间会得到改善"。

```
$ perf record -e cycles -b -- ./a.exe
[ perf record: Woken up 3 times to write data ]
[ perf record: Captured and wrote 0.535 MB perf.data (670 samples) ]
$ perf report -n --sort overhead,srcline_from,srcline_to -F +dso,symbol_from,symbol_to --stdio
# Samples: 21K of event 'cycles'
# Event count (approx.): 21440
# Overhead  Samples  Object  Source Sym  Target Sym  From Line  To Line
# ........  .......  ......  ..........  ..........  .........  .......
  51.65%      11074   a.exe   [.] bar     [.] bar      a.c:4      a.c:5
  22.30%       4782   a.exe   [.] foo     [.] bar      a.c:10     (null)
  21.89%       4693   a.exe   [.] foo     [.] zoo      a.c:11     (null)
   4.03%        863   a.exe   [.] main    [.] foo      a.c:21     (null)
```

从这个例子中，我们可以看到超过 50% 的已采取分支位于 bar 函数内，22% 的分支是来自 foo 到 bar 的函数调用，等等。请注意，perf 如何从 cycles 事件切换到分析 LBR 堆栈：只收集了 670 个样本，但每个样本都捕获了整个 LBR 堆栈。这为我们提供了 21440 个 LBR 条目（分支结果）进行分析。[99]

[99] perf 生成的报告头可能仍然让人困惑，因为它说 "21K of event cycles"。但这里有 "21K" 个 LBR 条目，而不是 "cycles"。

大多数情况下，仅从代码行和目标符号就可以确定分支的位置。然而，理论上，可以编写代码，在单行上写两个 `if` 语句。此外，展开宏定义时，所有展开的代码都归于相同的源行，这也是可能发生此类情况的另一个场景。这个问题不会完全阻止分析，只会使其稍微困难一些。为了消除两个分支的歧义，您可能需要自己分析原始 LBR 堆栈（请参阅 easyperf 博客上的示例：https://easyperf.net/blog/2019/05/06/Estimating-branch-probability[100]）。

使用分支记录，我们还可以找到一个“超块”（有时称为“超级块”），它是函数中一系列热门基本块的链，这些基本块不一定按照顺序排列，但它们是顺序执行的。因此，超块代表了函数、代码片段或程序的典型热路径。

6.4.3 分析分支预测错误率

由于每个记录中保存的附加信息中包含预测错误位，因此还可以知道热门分支的预测错误率。在这个例子中，我们使用了 LLVM 测试套件中 7-zip 基准的纯 C 代码版本。[101] `perf report` 的输出经过稍微修剪，以便更好地适应页面。以下用例在 Intel 和 AMD 上都支持。根据 ARM 的 BRBE 规范，它可以支持，但由于缺乏实现此扩展的处理器，无法验证。

```
$ perf record -e cycles -b -- ./7zip.exe b
$ perf report -n --sort symbol_from,symbol_to -F +mispredict,srcline_from,srcline_to --stdio
# Samples: 657K of event 'cycles'
# Event count (approx.): 657888
# Overhead  Samples  Mis  From Line  To Line  Source Sym  Target Sym
# ........  .......  ...  .........  .......  ..........  ..........
    46.12%   303391   N   dec.c:36   dec.c:40  LzmaDec     LzmaDec
    22.33%   146900   N   enc.c:25   enc.c:26  LzmaFind    LzmaFind
     6.70%    44074   N   lz.c:13    lz.c:27   LzmaEnc     LzmaEnc
     6.33%    41665   Y   dec.c:36   dec.c:40  LzmaDec     LzmaDec
```

在这个例子中，与函数 `LzmaDec` 相对应的行是我们特别关注的。按照上一节类似的分析，我们可以得出结论，`dec.c:36` 源行上的分支是基准测试中执行次数最多的分支。在 Linux `perf` 提供的输出中，我们可以看到两个与 `LzmaDec` 函数相对应的条目：一个带有 `Y` 字母，另一个带有 `N` 字母。将这两个条目一起分析，我们可以得到该分支的预测错误率。在这种情况下，我们知道 `dec.c:36` 行上的分支被正确预测了 `303391` 次（对应于 `N`），被错误预测了 `41665` 次（对应于 `Y`），因此预测率为 `88%`。

Linux `perf` 通过分析每个 LBR 条目并从中提取预测错误位来计算预测错误率。因此，对于每个分支，我们都知道它被正确预测的次数和错误预测的次数。同样，由于采样的性质，一些分支可能有一个 `N` 条目，但没有对应的 `Y` 条目。这可能意味着没有该分支被错误预测的 LBR 条目，但这并不意味着预测率是 `100%`。

6.4.4 机器码的精确计时

正如我们在英特尔 LBR 部分所展示的，从 Skylake 微架构开始，LBR 条目中有一个特殊的 周期计数 字段。这个附加字段指定了两个已采取分支之间经过的周期数。由于前一个 (N-1) LBR 条目中的目标地址是一个基本块 (BB) 的开始，而当前 (N) LBR 条目中的源地址是同一个基本块的最后一个指令，因此周期计数就是这个基本块的延迟。

这种类型的分析在 AMD 平台上不受支持，因为它们不会在 LBR 记录中记录周期计数。根据 ARM 的 BRBE 规范，它可以支持，但由于缺乏实现此扩展的处理器，无法验证。但是，英特尔支持它。这里是一个例子：

```
400618:   movb  $0x0, (%rbp,%rdx,1)    <= start of a BB
40061d:   add $0x1, %rdx
400621:   cmp $0xc800000, %rdx
```

[100] Easyperf: 估计分支概率 - https://easyperf.net/blog/2019/05/06/Estimating-branch-probability

[101] LLVM 测试套件 7zip 基准 - https://github.com/llvm-mirror/test-suite/tree/master/MultiSource/Benchmarks/7zip

```
400628:   jnz 0x400644                <= end of a BB
```

假设我们在 LBR 堆栈中有两个条目：

```
FROM_IP   TO_IP    Cycle Count
...       ...      ...
40060a    400618    10
400628    400644     5          <== LBR TOS
```

根据这些信息，我们知道从偏移量 400618 开始执行的基本块以 5 个周期执行了一次。如果我们收集足够多的样本，我们可以绘制该基本块延迟的概率密度图。

图 41 展示了这样的图表示例。它是通过分析所有满足上述规则的 LBR 条目编译而成的。读取图表的方法如下：它告诉我们给定延迟值出现的比率。例如，大约 2% 的时间测量到基本块延迟正好为 100 周期，14% 的时间测量到 280 周期，我们从未见过 150 到 200 周期之间的数值。另一种解读方式是：根据收集的数据，如果您要测量某个基本块的延迟，看到特定延迟的概率是多少?

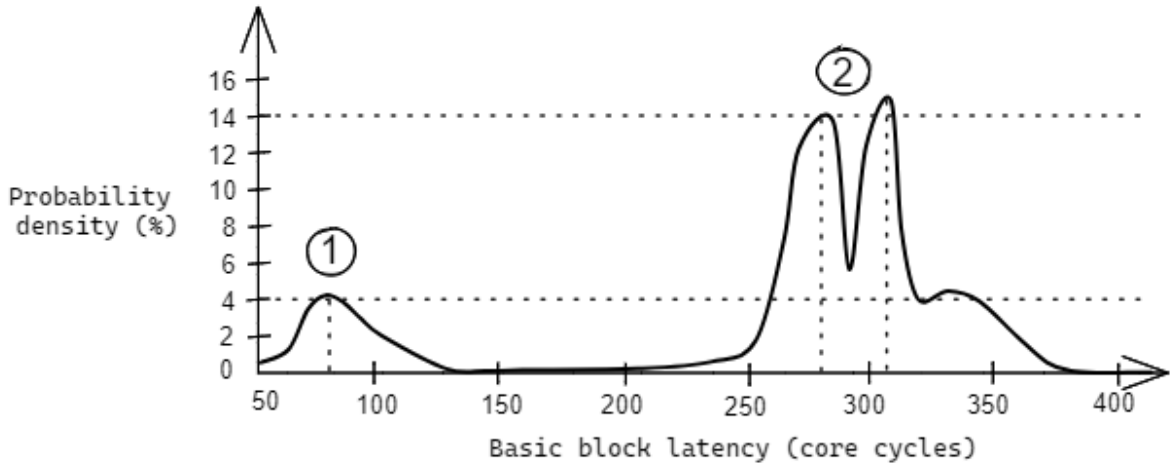


Figure 41: 基本块延迟的概率密度图，基本块起始地址为 0x400618

我们可以清楚地看到两个峰值：一个较小的峰值大约在 80 周期①，两个更大的峰值在 280 和 305 周期②。该块从一个不适合 CPU L3 缓存的大数组中进行非顺序加载，因此基本块的延迟很大程度上取决于此加载。基于图表，我们可以得出结论，第一个峰值①对应于 L3 缓存命中，第二个峰值②对应于 L3 缓存未命中，其中加载请求一直到主内存。

这些信息可以用于对该基本块进行细粒度调整。此示例可能受益于内存预取，我们将在 Section 8.2 中讨论。此外，周期计数信息可用于循环迭代的计时，其中每个循环迭代都以一个已采取的分支（后向边缘 (back edge)）结束。

在适当的性能分析工具支持到位之前，构建类似于图 41 的概率密度图需要手动解析原始 LBR 转储。有关如何执行此操作的示例，请参见 easyperf 博客: https://easyperf.net/blog/2019/04/03/Precise-timing-of-machine-code-with-Linux-perf[102]。幸运的是，在较新的 Linux perf 版本中，获取这些信息要容易得多。以下示例直接使用 Linux perf 在我们之前介绍的 LLVM 测试套件中相同的 7-zip 基准测试上演示了这种方法：

```
$ perf record -e cycles -b -- ./7zip.exe b
$ perf report -n --sort symbol_from,symbol_to -F +cycles,srcline_from,srcline_to --stdio
# Samples: 658K of event 'cycles'
```

[102] Easyperf: 为任意基本块的延迟构建概率密度图 - https://easyperf.net/blog/2019/04/03/Precise-timing-of-machine-code-with-Linux-perf.

```
# Event count (approx.): 658240
# Overhead  Samples  BBCycles  FromSrcLine  ToSrcLine
# ........  .......  ........  ...........  .........
     2.82%   18581       1      dec.c:325    dec.c:326
     2.54%   16728       2      dec.c:174    dec.c:174
     2.40%   15815       4      dec.c:174    dec.c:174
     2.28%   15032       2      find.c:375   find.c:376
     1.59%   10484       1      dec.c:174    dec.c:174
     1.44%   9474        1      enc.c:1310   enc.c:1315
     1.43%   9392       10      7zCrc.c:15   7zCrc.c:17
     0.85%   5567       32      dec.c:174    dec.c:174
     0.78%   5126        1      enc.c:820    find.c:540
     0.77%   5066        1      enc.c:1335   enc.c:1325
     0.76%   5014        6      dec.c:299    dec.c:299
     0.72%   4770        6      dec.c:174    dec.c:174
     0.71%   4681        2      dec.c:396    dec.c:395
     0.69%   4563        3      dec.c:174    dec.c:174
     0.58%   3804       24      dec.c:174    dec.c:174
```

请注意，我们添加了 `-F +cycles` 选项以在输出中显示周期计数（“BBCycles”列）。为适应页面大小，我们删除了几行无关紧要的 `perf report` 输出。让我们关注源代码和目标代码都是 `dec.c:174` 的行，输出中有七行这样的行。在源代码中，行 `dec.c:174` 展开了一个包含自包含分支的宏。这就是为什么源代码和目标代码恰好位于同一行的原因。

Linux `perf` 首先按开销对条目进行排序，因此我们需要手动过滤我们感兴趣的分支的条目。幸运的是，它们可以通过 `grep` 命令轻松过滤。事实上，如果我们过滤它们，我们将得到以这个分支结尾的基本块的延迟分布，如表 7 所示。这些数据可以绘制成一个类似于图 41 的图表。

Table 7: 基本块延迟的概率密度。

周期	样本数	概率密度
1	10484	17.0%
2	16728	27.1%
3	4563	7.4%
4	15815	25.6%
6	4770	7.7%
24	3804	6.2%
32	5567	9.0%

以下是我们如何解释这些数据：从所有收集的样本中，17% 的时间基本块的延迟为 1 个周期，27% 的时间为 2 个周期，等等。请注意，分布主要集中在 1 到 6 个周期，但也有第二个模式，延迟高得多，为 24 和 32 个周期，这可能对应于分支预测错误惩罚。分布中的第二个模式占所有样本的 15%。

这个例子表明，不仅可以绘制微型基准测试的基本块延迟，还可以绘制实际应用程序的基本块延迟。目前，LBR 是英特尔系统上最精确的周期级计时信息源。

6.4.5 估计分支结果概率

在后面的 Chapter 11 部分，我们将讨论代码布局对性能的重要性。进一步说，以透底方式放置热路径[103] 通常可以提高程序性能。知道某个分支最常见的执行结果可以让开发人员和编译器做出更好的优化决策。例如，如果一个分支有 99% 的时间被执行，我们可以尝试反转条件将其转换为未执行分支。

LBR 可以让我们在不检测代码的情况下收集这些数据。分析结果将为用户提供条件真假结果之间的比率，即分支被执行和未执行的次数。这一特性在分析间接跳转（switch 语句）和间接调用（虚函数调用）时尤其有用。您可以在 easyperf 博客上找到实际应用示例：https://easyperf.net/blog/2019/05/06/Estimating-branch-probability[104]。

6.4.6 提供编译器反馈数据

我们将在后面的 Section 11.7 部分讨论基于优化分析文件的优化 (PGO)，这里先简要提一下。分支记录机制可以为优化编译器提供优化分析文件反馈数据。想象一下，我们可以将我们在前面部分发现的所有数据反馈给编译器。在某些情况下，这些数据无法使用传统的静态代码检测工具获得，因此分支记录机制不仅因开销更低而成为更好的选择，而且还能提供更丰富的优化分析文件数据。依赖硬件 PMU 收集数据进行的 PGO 工作流程越来越流行，一旦 AMD 和 ARM 的支持成熟，可能会迅速发展。

6.5 基于硬件的采样功能

主要 CPU 供应商提供了一系列附加功能来增强性能分析。由于 CPU 供应商以不同的方式处理性能监控，因此这些功能不仅在调用方式上存在差异，而且在功能上也存在差异。在 Intel 处理器中，它被称为处理器事件采样 (PEBS)，首次引入于 NetBurst 微架构。AMD 处理器上类似的功能称为指令采样 (IBS)，从 AMD Opteron 家族 (10h 代) 核心开始可用。接下来，我们将更详细地讨论这些功能，包括它们的相似之处和不同之处。

6.5.1 英特尔平台上的 PEBS

与最后分支记录类似，PEBS 用于在分析程序时捕获每个收集到的样本的额外数据。当性能计数器配置为 PEBS 时，处理器会保存一组具有定义格式的额外数据，称为 PEBS 记录。英特尔 Skylake CPU 的 PEBS 记录格式如图 42 所示。记录包含通用寄存器状态 (`EAX`, `EBX`, `ESP` 等)、`EventingIP`, `Data Linear Address` 和稍后将讨论的 延迟值。PEBS 记录的内容布局因不同的微架构而异，请参阅 [Intel, 2023b, 第 3B 卷，第 20 章 性能监控]。

从 Skylake 架构开始，PEBS 记录已经增强，可以收集 XMM 寄存器和 LBR 记录。格式已经重新组织，将字段分组为基本组、内存组、GPR 组、XMM 组和 LBR 组。性能分析工具可以选择感兴趣的数据组，从而减小记录大小并降低记录生成延迟。默认情况下，PEBS 记录只包含基本组。

使用 PEBS 的一个显著优点是与基于中断的常规采样相比，采样开销更低。回想一下，当计数器溢出时，CPU 会生成中断来收集一个样本。频繁地生成中断并让分析工具本身在中断服务例程中捕获程序状态是非常昂贵的，因为它涉及操作系统交互。

另一方面，PEBS 维护了一个缓冲区，用于临时存储多个 PEBS 记录。假设我们使用 PEBS 对加载事件进行采样。当性能计数器配置为 PEBS 时，计数器溢出条件不会触发中断，而是会激活 PEBS 机制。该机制将捕获下一个加载，捕获一个新的记录并将其存储在专用的 PEBS 缓冲区区域。该机制还会清除计数器溢出状态并重新加载计数器以初始值。只有当专用缓冲区已满时，处理器才会引发中断，缓冲区才会刷新到内存。这种机制通过减少中断触发次数来降低采样开销。

Linux 用户可以通过执行 `dmesg` 检查 PEBS 是否已启用：

```
$ dmesg | grep PEBS
```

[103] 也就是说，当热分支没有被执行时。

[104] Easyperf: 估计分支概率 - https://easyperf.net/blog/2019/05/06/Estimating-branch-probability

Byte Offset	Field	Byte Offset	Field
00H	R/EFLAGS	68H	R11
08H	R/EIP	70H	R12
10H	R/EAX	78H	R13
18H	R/EBX	80H	R14
20H	R/ECX	88H	R15
28H	R/EDX	90H	Applicable Counter
30H	R/ESI	98H	Data Linear Address
38H	R/EDI	A0H	Data Source Encoding
40H	R/EBP	A8H	Latency value (core cycles)
48H	R/ESP	B0H	EventingIP
50H	R8	B8H	TX Abort Information (Section 18.3.6.5.1)
58H	R9	C0H	TSC
60H	R10		

Figure 42: PEBS 记录格式适用于第六代、第七代和第八代英特尔酷睿处理器家族。© Image from [Intel, 2023b, Volume 3B, Chapter 20].

```
[    0.113779] Performance Events: XSAVE Architectural LBR, PEBS fmt4+-baseline,
AnyThread deprecated, Alderlake Hybrid events, 32-deep LBR, full-width counters, Intel PMU driver.
```

对于 LBR，Linux perf 会在每个收集到的样本中转储整个 LBR 堆栈内容。因此，可以分析由 Linux perf 收集的原始 LBR 转储。但是，对于 PEBS，Linux `perf` 不会像 LBR 那样导出原始输出。相反，它会处理 PEBS 记录并仅提取根据特定需求的数据子集。因此，无法使用 Linux `perf` 访问原始 PEBS 记录的集合。但是，Linux `perf` 提供了一些从原始样本处理过的 PEBS 数据，可以通过 `perf report -D` 访问。要转储原始 PEBS 记录，可以使用 `pebs-grabber`: https://github.com/andikleen/pmu-tools/tree/master/pebs-grabber[105] 工具。

6.5.2 AMD 平台上的 IBS

指令采样 (IBS) 是 AMD64 处理器的一项功能，可用于收集与指令提取和指令执行相关的特定指标。AMD 处理器的处理器流水线由两个独立的阶段组成：一个前端阶段负责提取 AMD64 指令字节，一个后端阶段负责执行 “ops”。由于这些阶段在逻辑上是分开的，因此存在两个独立的采样机制：IBS Fetch 和 IBS Execute。

- IBS Fetch 监控流水线的前端，并提供有关 ITLB (命中或未命中)、I-cache (命中或未命中)、获取地址、获取延迟等信息。
- IBS Execute 监控流水线的后端，通过跟踪单个 op 的执行来提供关于指令执行行为的信息。例如：分支 (是否被执行，是否被预测)、加载/存储 (D-cache 和 DTLB 中的命中或未命中，线性地址，加载延迟)。

PMC 和 IBS 在 AMD 处理器之间存在一些重要差异。PMC 计数器是可编程的，而 IBS 则类似于固定计数器。IBS 计数器只能用于监控启用或禁用，无法针对任何选择性事件进行编程。IBS Fetch 和 Execute 计数器可以独立启用/禁用。使用 PMC 时，用户必须提前决定要监控哪些事件。使用 IBS 时，会为每个采样的指令收集丰富的数据，然后由用户分析他们感兴趣的数据部分。IBS 选择并标记要监控的指令，然后捕获该指令执行期间引起的微架构事件。有关 Intel PEBS 和 AMD IBS 的更多详细比较，请参见 [Sasongko et al., 2023]。

由于 IBS 被集成到处理器流水线中并作为固定事件计数器，因此样本收集开销最小。分析器需要处理 IBS 生成的数据，这些数据可能非常庞大，具体取决于采样间隔、配置的线程数、是否配置 Fetch/Execute 等。在 Linux 内核版本 6.1 之前，IBS 总是为所有内核收集样本。这个限制会导致巨大的数据收集和处理开销。从内核 6.2 开始，Linux perf 只支持为配置的内核收集 IBS 样本。

[105] PEBS 抓取器工具 - https://github.com/andikleen/pmu-tools/tree/master/pebs-grabber。需要 root 权限。

IBS 由 Linux perf 和 AMD uProf 分析器支持。以下是收集 IBS 执行和获取样本的示例命令：

```
$ perf record -a -e ibs_op/cnt_ctl=1,l3missonly=0/ -- benchmark.exe
$ perf record -a -e ibs_fetch/l3missonly=1/ -- benchmark.exe
$ perf report
```

在上一个命令中，`cnt_ctl=0` 表示计数时钟周期，`cnt_ctl=1` 表示在间隔期间计数已分配的 ops；`l3missonly=1` 只保留具有 L3 未命中的样本。请注意，在上述两个命令中，都使用了 `-a` 选项来为所有内核收集 IBS 样本，否则 `perf` 在 Linux 内核 6.1 上无法收集样本。从 6.2 版本开始，除非您想要为所有内核收集 IBS 样本，否则不再需要 `-a` 选项。`perf report` 命令将显示类似于常规 PMU 事件的与函数和源代码行关联的样本，但会提供我们稍后讨论的附加功能。

6.5.3 ARM 平台上的 SPE

Arm Statistical Profiling Extension (SPE) 是一项架构功能，旨在增强 Arm CPU 内的指令执行性能分析。自 2019 年推出的 Neoverse N1 核心以来，这项功能就可用。SPE 功能扩展被指定为 Armv8-A 架构的一部分，从 Arm v8.2 起提供支持。与其他解决方案相比，SPE 更类似于 AMD IBS，而不是 Intel PEBS。类似于 IBS，SPE 与通用性能监测器计数器 (PMC) 分开，但它只有一个机制，而不是两种类型的 IBS（获取和执行）。

SPE 采样过程内置于指令执行流水线中。样本收集仍然基于可配置的间隔，但操作是根据统计信息选择的。每个采样的操作都会生成一个样本记录，其中包含有关此操作执行的各种数据。SPE 记录保存指令地址，负载和存储访问的数据的虚拟和物理地址，数据访问的来源（缓存或 DRAM）以及时间戳，以与系统中其他事件进行关联。此外，它还可以提供各种流水线阶段的延迟，例如发出延迟（从调度到执行）、转换延迟（虚拟到物理地址转换的周期数）和执行延迟（功能单元中负载/存储的延迟）。白皮书 [Arm, 2023b] 更详细地描述了 ARM SPE，并展示了一些使用它的优化示例。

类似于 Intel PEBS 和 AMD IBS，ARM SPE 有助于减少采样开销并支持更长的收集。除此之外，它还支持样本记录的后过滤，这有助于减少存储所需内存。

SPE 分析已在 Linux `perf` 工具中启用，可以使用以下方式：

```
$ perf record -e arm_spe_0/<controls>/ -- test_program
$ perf report --stdio
$ spe-parser perf.data -t csv
```

其中 `<controls>` 允许您可选地指定收集的各种控件和过滤器。`perf report` 将根据用户使用 `<controls>` 选项所要求的内容提供通常的输出。`spe-parser`[106] 是由 ARM 工程师开发的工具，用于解析捕获的 perf 记录数据并将所有 SPE 记录保存到 CSV 文件中。

6.5.4 精确事件

现在我们已经介绍了高级采样功能，让我们讨论一下 **如何** 使用它们来改善性能分析。我们将从精确事件的概念开始。

性能分析的一个主要问题是精确地定位导致特定性能事件的指令。正如 Section 5.5 中所讨论的，基于中断的采样基于计数特定性能事件并等待其溢出。当溢出中断发生时，处理器需要一段时间停止执行并标记导致溢出的指令。对于现代复杂的乱序 CPU 架构来说，这一点尤其困难。

它引入了滑动的概念，滑动定义为导致事件的 IP (指令地址) 与事件被标记的 IP 之间的距离。滑动使得难以发现导致性能问题的指令。考虑一个具有大量缓存未命中的应用程序，其热门汇编代码如下所示：

[106] ARM SPE 解析器 - https://gitlab.arm.com/telemetry-solution/telemetry-solution

```
; load1
; load2
; load3
```

分析器可能会将 load3 标记为导致大量缓存未命中的指令，而实际上，真正的罪魁祸首是 load1。对于高性能处理器，这种滑动可能数百条处理器指令。这通常会让性能工程师感到非常困惑。有兴趣的读者可以访问 Intel 开发者专区网站: https://software.intel.com/en-us/vtune-help-hardware-event-skid[107] 了解更多关于此类问题基础原因的信息。

通过让处理器本身存储指令指针（以及其他信息）可以缓解滑移问题。使用 Intel PEBS 时，PEBS 记录中的 EventingIP 字段指示导致事件的指令。这通常仅适用于受支持事件的一个子集，称为“精确事件”。可以在 [Intel, 2023b, Volume 3B, Chapter 20 Performance Monitoring] 中找到特定微架构的精确事件完整列表。有关使用 PEBS 精确事件缓解滑移的示例，请参见 easyperf 博客: https://easyperf.net/blog/2018/08/29/Understanding-performance-events-skid.[108]

以下是 Skylake 微架构的精确事件列表：

```
INST_RETIRED.*          OTHER_ASSISTS.*      BR_INST_RETIRED.*       BR_MISP_RETIRED.*
FRONTEND_RETIRED.*      HLE_RETIRED.*        RTM_RETIRED.*           MEM_INST_RETIRED.*
MEM_LOAD_RETIRED.*      MEM_LOAD_L3_HIT_RETIRED.*
```

其中 * 表示组内所有子事件都可以配置为精确事件。

使用 Intel 平台上 Linux perf 的用户应在上述列出的事件之一中添加 pp 后缀以启用精确标记：

```
$ perf record -e cycles:pp -- ./a.exe
```

对于 AMD IBS 和 ARM SPE，所有收集的样本在设计上都是精确的，因为硬件会捕获确切的指令地址。事实上，它们都以非常相似的方式工作。每当溢出发生时，该机制将导致溢出的指令保存到专用缓冲区中，然后由中断处理程序读取。由于地址被保留，因此 IBS 和 SPE 样本与指令的关联是精确的。

精确事件为性能工程师提供了便利，因为它们有助于避免误导性数据，这些数据经常让初学者甚至高级开发人员感到困惑。TMA 方法论依赖精确事件来定位低效执行发生的源代码确切行。

6.6 分析内存访问

内存访问是许多应用程序性能的关键因素。PEBS 和 IBS 都能够收集程序中内存访问的详细信息。例如，您不仅可以对加载进行采样，还可以收集它们的目标地址和访问延迟。请记住，这并不跟踪所有存储和加载。否则，开销会太大。相反，它只分析大约每 10 万次访问中的一个访问。您可以自定义每秒需要多少个样本。通过足够的样本收集，可以提供内存访问的准确统计图片。

在 PEBS 中，允许实现此功能的功能称为数据地址分析 (DLA)。为了提供有关采样加载和存储的更多信息，它使用 PEBS 设施内的“Data Linear Address”和“Latency Value”字段（参见图 42 ）。如果性能事件支持 DLA 功能并且 DLA 已启用，处理器将转储所采样内存访问的内存地址和延迟。您还可以过滤延迟高于某个阈值的内存访问。这对于查找可能成为许多应用程序性能瓶颈的长延迟内存访问非常有用。

使用 IBS Execute 和 ARM SPE 采样，您还可以深入分析应用程序执行的内存访问。一种方法是转储收集的样本并手动处理它们。IBS 会保存确切的线性地址、其延迟、访问来自何处（缓存或 DRAM）、以及它是否在 DTLB 中命中或未命中。SPE 可用于估计内存子系统组件的延迟和带宽、估计单个加载/存储的内存延迟等等。

[107] 硬件事件滑移 - https://software.intel.com/en-us/vtune-help-hardware-event-skid
[108] 性能滑移 - https://easyperf.net/blog/2018/08/29/Understanding-performance-events-skid

这些扩展最重要的用例之一是检测真实共享和虚伪共享，我们将在 Section 13.7 中讨论。Linux `perf c2c` 工具大量依赖所有三种机制（PEBS、IBS 和 SPE）来查找可能遇到真实/虚伪共享的竞争内存访问：它匹配不同线程的加载/存储地址，并检查命中是否发生在由其他线程修改的缓存行中。

问题和练习

1. TMA 性能方法论的四个一级类别是什么?
2. HW 事件采样的优势是什么?
3. 什么是性能事件滑移?
4. 研究用于开发/基准测试的机器内部 CPU 上可用的性能分析功能。

AI 回答：

1. **TMA 性能方法论的四个一级类别**：
   - **Front End Bound (FE bound)**：表示前端瓶颈，即 CPU 前端（如指令获取和解码）的瓶颈。
   - **Back End Bound (BE bound)**：表示后端瓶颈，即 CPU 后端（如执行单元）的瓶颈。
   - **Retiring**：表示退役，即 CPU 执行指令并将其结果写回寄存器的能力。
   - **Bad Speculation**：表示错误推测，即分支预测错误导致的性能损失。
2. **HW 事件采样的优势**：
   - **降低采样开销**：通过在专用缓冲区中存储多个样本，减少了因软件中断而产生的开销。
   - **提高精确度**：精确事件能够精确地定位导致特定性能事件的指令，减少了性能分析中的滑移问题。
   - **支持长时间运行的分析**：由于采样开销的降低，可以进行更长时间的性能数据收集，而不会显著影响程序的运行。
   - **提供丰富的数据**：采样功能可以捕获执行过程中的详细信息，如寄存器状态、内存访问地址和延迟等。
3. **性能事件滑移**：
   - 性能事件滑移（Hardware Event Skid）是指在现代乱序执行的 CPU 中，由于指令流水线的复杂性，导致性能事件（如缓存未命中）的实际发生位置与软件分析工具标记的位置之间的偏差。这种偏差使得分析工具难以准确地确定导致性能问题的指令。
4. **研究用于开发/基准测试的机器内部 CPU 上可用的性能分析功能**：
   - **Intel**：提供了 TMA、LBR、PEBS 等特性，支持精确事件采样，以及处理器跟踪（PT）功能。
   - **AMD**：提供了 LBR、IBS（Instruction Based Sampling）等特性，支持精确事件采样。
   - **ARM**：提供了 BRBE（Branch Record Buffer Extension）、SPE（Statistical Profiling Extension）等特性，支持精确事件采样。
   - **Linux perf**：是一个性能分析工具，支持上述硬件特性，可以用于收集和分析性能数据。
   - **AMD uProf**：AMD 提供的性能分析工具，支持 IBS 等特性。
   - **其他工具**：如 VTune Profiler、toplev 等，这些工具可以利用硬件性能监控特性来进行性能分析。

开发者和性能工程师可以利用这些功能来识别和解决应用程序中的性能瓶颈。

章节总结

- 仅当所有高级性能问题都已修复后，才建议使用硬件功能进行低级微调。调整设计不良的算法是时间上的浪费。一旦消除所有主要性能问题，就可以使用 CPU 性能监控功能来分析和进一步调整应用程序。
- 自顶向下的微架构分析 (TMA) 方法是一种非常强大且易学的方法，即使是经验不足的开发人员也可以轻松使用它来识别程序对 CPU 微架构的无效使用。TMA 是一个迭代过程，包括多个步骤，其中包括描述工作负载和定位源代码中瓶颈发生的确切位置。我们建议 TMA 应该成为每次低级调整工作的起点之一。
- 分支记录机制（例如 Intel 的 LBR、AMD 的 LBR 和 ARM 的 BRBE）在执行程序的同时连续记录最近的分支结果，从而导致最小的减速。这些设施的主要用途之一是收集调用堆栈。此外，它们还有助于识别热点分支、预

测错误率并实现机器代码的精确计时。

- 现代处理器通常提供基于硬件的采样功能，用于高级性能分析。此类功能通过将多个样本存储到专用缓冲区而不使用软件中断来降低采样开销。它们还引入了“精确事件”，能够精确定位导致特定性能事件的确切指令。此外，还有一些其他不太重要的用例。此类基于硬件的采样功能的示例实现包括 Intel 的 PEBS、AMD 的 IBS 和 ARM 的 SPE。
- Intel 处理器跟踪 (PT) 是一项 CPU 功能，通过以高度压缩的二进制格式编码数据包来记录程序执行，这些数据包可用于重建带有每个指令时间戳的执行流。PT 具有广泛的覆盖范围和相对较小的开销。其主要用途是事后分析和查找性能问题根本原因。Intel PT 功能在附录 D 中介绍。基于 ARM 架构的处理器也具有称为 ARM CoreSight 的跟踪功能[109]，但它主要用于调试而非性能分析。

性能分析器利用本章介绍的硬件功能来实现许多不同类型的分析。

[109] ARM CoreSight - https://developer.arm.com/ip-products/system-ip/coresight-debug-and-trace